

HIGH-LEVEL DESIGN (HLD) DOCUMENT AND CONTRACT

Educational Application Development | ET 617

Project Title: AI-Assisted Spoken Tutorial Video Generator

Client: EduPyramids

Student Team: Group 10 - Sameer Gupta(23b2509,Team Lead), Daksh(23b2503), Sumedh Taru(23b2487), Sanidhya Katiyar(23b2244)

Version: 2.0

Date: 19 August 2026

Document Status: Draft

1. Project Information

Item	Details
Project Title	AI-Assisted Spoken Tutorial Video Generator
Client	EduPyramids
Client Contact	Saisudha - 7259400337
Student Team	Sameer Gupta(23b2509,Team Lead), Daksh(23b2503), Sumedh Taru(23b2487), Sanidhya Katiyar(23b2244)
Primary Users	Students
Application Type	Web application
Target Platform	Desktop web - Chrome, Edge
HLD Version	2.0
Date	19 August 2026

2. Project Overview

2.1 Problem Statement

Spoken Tutorials are step-by-step instructional videos in which narration is tightly coupled to on-screen software actions. The learner is expected to pause, perform the demonstrated action, and resume. Because of this coupling, production is highly manual: an author writes a timed script under a strict pedagogy (short, simple sentences, an 80-character-per-sentence limit in English), performs the software actions, records narration, edits, and exports.

The bottleneck the Client has identified is multilingual production. The visual track is authored once in English and reused across all languages; only the audio is replaced. Indian languages, however, do not compress to the same duration as the English source. For Example - Tamil in particular expands substantially for the same semantic content. To fit the fixed timed script, dubbing artists speak faster than natural pace. The result is narration that sounds rushed, degrades comprehension for the target learner (a learner who may be as young as Class V), and forces repeated re-recording.

There is a second, structural cost: production is entirely manual, so every additional language, every script revision and every software-version update multiplies human effort rather than reusing prior work.

2.2 Proposed Solution

An AI-assisted authoring tool that takes a structured Spoken Tutorial timed script and produces an editable multilingual tutorial project. The system parses the script into narration segments and action cues, generates narration using Sarvam AI text-to-speech (with optional voice cloning), and critically treats duration as a first-class constraint rather than an after-the-fact problem.

Duration is handled in the following way: translation is generated under a per-segment time budget: the system predicts the spoken duration of a candidate translation and iteratively rephrases it until it fits, so the language itself absorbs most of the expansion instead of the speaking rate.

The tool is explicitly an assistant, not a replacement. Every generated artefact segmentation, translation, narration, timeline - is inspectable and overridable, and the final output is an editable project file that opens in a standard video editor rather than a locked-down rendered file.

2.3 Project Objectives

The project aims to:

1. Convert a structured Spoken Tutorial script into a segmented, machine-readable timeline with narration segments and action cues separated and time-budgeted.
2. Generate natural-sounding narration in English and at least two Indian languages using Sarvam AI, with speaking rate held within $\pm 10\%$ of the reference pace.

3. Reduce audio–video drift to a measurable, verifiable threshold (≤ 250 ms per segment) and report it, instead of relying on manual review.
4. Export an editable project file (OpenTimelineIO / Kdenlive XML) plus per-segment audio stems and subtitles, so the Client's existing editorial workflow is preserved.
5. Demonstrate a measurable reduction in manual production effort for a multilingual tutorial relative to the Client's current process.

2.4 Deployment Context

Intended Context:

EduPyramids content production team; secondary deployment as an internal tool for partner institutions producing Spoken Tutorial content.

Expected Users:

Testing team: a small internal team.

Beneficiaries: the Client reports 1 crore+ learners trained to date, with a growing school-sector user base.

Technology Context:

Authors work on desktop machines with reliable broadband. AI inference is cloud-based (Sarvam AI).

Rendering is server-side and asynchronous. Learners consume the finished videos on low-end devices and intermittent connectivity, which constrains output file size and codec choice, but not the authoring tool itself.

3. User Personas

3.1 Primary Persona

Persona: Lakshmi R. - EduPyramids content author

Example: Mathematics Teacher

Profile:

Age 26-40. Employed by or contracted to the Client. Strong in the Spoken Tutorial pedagogy and the 80-character timed-script convention; comfortable with screen recording and a desktop video editor; not a programmer and not an AI practitioner

Goals:

Produce a complete tutorial in multiple languages without re-recording; keep narration aligned to on-screen actions; preserve pedagogical quality across languages.

Needs:

A predictable script-to-video path; visibility into which segments will not fit their time budget, before synthesis; the ability to override any automated decision; output that opens in the editor already in use

Pain Points:

- Manual re-recording for every language and every script revision
- dubbing artists rushing to fit the timed script
- no early warning that a translated segment will overrun

Technology Context:

- Device: desktop/laptop
- Connectivity: Institutional broadband
- Digital literacy: High for media tools, low-to-moderate for AI tooling and command-line workflows

Motivations:

Reaching learners in their own language at the quality bar the pedagogy demands, removing repetitive production work that does not use their expertise

Constraints:

Fixed publication schedules; no engineering support for day-to-day operation; must stay within the existing Spoken Tutorial format and pedagogy

Success Criteria:

A multilingual tutorial produced in materially less time than the current process, with narration that a reviewer judges unrushed and with no manual re-syncing required before export

3.2 Secondary Persona

Persona: Karthik S. - Language reviewer / dubbing artist

Profile:

Native speaker with subject familiarity, reviews translated narration for accuracy, register, and pronunciation of technical terms

Goals:

Ensure the translation is faithful and natural; ensure technical terms (e.g., "list", "variable", "compile") are rendered consistently across the tutorial series

Needs:

- Side-by-side source and translated segments
- A pronunciation lexicon that persists across projects
- The ability to correct a segment and re-synthesise only that segment

Pain Points:

- Being asked to compress meaning to fit a duration that was never designed for the language
- Inconsistent terminology between tutorials
- Re-recording an entire file because of one term

Technology Context:

- Device: [desktop/laptop]
- Connectivity: [broadband]

Motivations:

Professional pride in the quality of the regional-language track

Constraints:

Reviews are time boxed; works across several tutorials at once

3.3 Additional Personas

Persona	Role	Primary Goal	Main Need
QA Reviewer	Production quality check	Approve or reject a language track before publication	An objective sync/rate report, not just subjective viewing
Platform Administrator	Operations	Keep API credentials, quotas, and users under control	Key management, per-project usage visibility, role control
Learner (indirect)	Class V – college/working professional	Follow the tutorial and complete each step successfully	Narration paced slowly enough to act on, in their own language

4. User Roles & Permissions

Define what each type of user can access and modify.

Feature	Author	Language Reviewer	QA Reviewer	Admin
Log in	✓	✓	✓	✓
Create project/upload script	✓			✓
Edit script segments & time budgets	✓			
Edit translated text/lexicon	✓	✓		
Trigger narration synthesis	✓	✓ (single segment)		✓
Trigger full render	✓			✓
View sync/rate QA report	✓	✓	✓	✓
Approve or reject a language track			✓	✓
Export editable project & stems	✓		✓	✓
Manage voice profiles & consent records	✓			✓
Manage users, API keys, quotas				✓

Note:

Approval is deliberately separated from authoring: the user who generates a track cannot sign it off. Language Reviewers can re-synthesise a single segment but not queue a full render, because render jobs consume paid API and compute quota. Only an Admin can register a cloned voice, and registration requires an attached consent record

5. User Journeys & Core Workflows

5.1 Primary User Journey

Describe the main workflow from the user's perspective.

Workflow:

Step 1: [Author creates a project and uploads the English timed script (.docx / .txt) and the base screen recording, if one already exists]

↓

Step 2: [System parses the script into narration segments and action cues, validates it against the pedagogy rules (sentence length, one instruction per sentence), and flags violations]

↓

Step 3: [Author reviews the segment list, fixes flagged segments, and confirms the per-segment time budget derived from the timed script]

↓

Step 4: [Author selects target languages and a voice profile; system translates each segment under its duration budget, predicting spoken duration and rephrasing until the segment fits]

↓

Step 5: [System synthesises narration via Sarvam AI, force-aligns the generated audio against the text, and computes per-segment drift and speaking rate]

↓

Step 6: [System builds a per-language timeline: narration is anchored to action cues, and residual slack is absorbed as short holds at non-action gaps only]

↓

Step 7: Author previews the language track in the browser and reads the QA report (max drift, mean rate, segments over budget)

↓

Step 8: Author corrects any segment and re-runs only the affected segment; QA Reviewer approves the track

↓

Step 9: System exports an editable project file, per-segment WAV stems, subtitles and a rendered preview

Primary Workflow Description

In prose: the author's work moves from recording to reviewing. Instead of performing and re-performing narration, the author supplies a script and adjudicates machine proposals - accepting a rewrite, correcting a term, nudging a hold. The system never silently changes meaning: a rewrite that changes a segment is surfaced as a suggestion with the original alongside it. Everything that leaves the system is editable, so an author who disagrees with any automated decision can fix it downstream in their own editor rather than fighting the tool

5.2 Secondary User Journey

Workflow:

A Language Reviewer opens an assigned track, sees source and translated segments side by side with the predicted and actual durations, corrects wording or adds a term to the pronunciation lexicon, and re-synthesises that segment alone. The timeline recomputes locally, the QA report updates, and the reviewer marks the track ready for QA. No full re-render is required for a single-segment correction - this is the main reason segment-level artefacts are versioned independently

5.3 Key Use Cases

Use Case	User	Description	Expected Outcome
UC-01	Author	Upload a timed script and obtain a validated segment list	Segments, action cues and time budgets extracted; rule violations flagged with segment IDs
UC-02	Author	Generate a Tamil track from the English script	Translated, duration-fitted narration with drift and rate within thresholds
UC-03	Language Reviewer	Correct one mistranslated segment and re-synthesise it	Only that segment is regenerated; timeline and QA report update automatically
UC-04	Author	Register and use a cloned voice for a language	Narration generated in the cloned voice, with a consent record attached to the voice profile
UC-05	QA Reviewer	Review the sync report and approve a track	Track marked approved; approval recorded with user, timestamp, and metrics
UC-06	Author	Export the approved track for editing	Editable project file, WAV stems, SRT, and preview render downloaded

Use Case	User	Description	Expected Outcome
UC-07	Admin	Switch the TTS provider from ElevenLabs to Sarvam	Provider changed at configuration level; no change to project data or exports

6. Functional Architecture

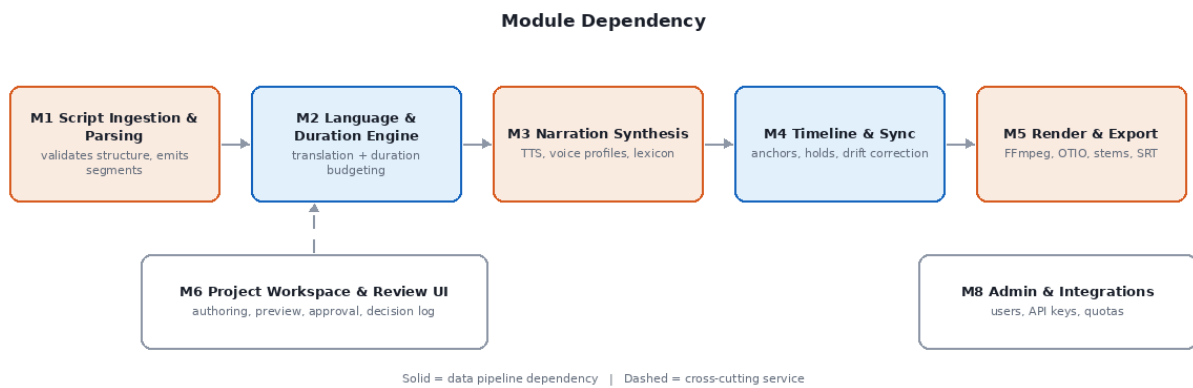
6.1 Major System Modules

The application will consist of the following major modules:

1. **Module 1 - Script Ingestion & Parsing**
 - **Purpose:** Turn an authored script into structured, validated data.
 - **Main functionality:** Accepts .docx/.txt; separates narration from action cues; extracts or derives per-segment time budgets; validates against pedagogy rules (sentence length, one instruction per sentence) and reports violations by segment ID.
2. **Module 2 - Language & Duration Engine**
 - **Purpose:** Translate to fit the time, instead of making the speaker rush.
 - **Main functionality:** Estimates spoken duration per language from a calibrated rate model; requests a duration-constrained translation from an LLM; iterates rephrasing until predicted duration $\leq$ budget or the iteration cap is hit; escalates unfittable segments to the author.
3. **Module 3 - Narration Synthesis**
 - **Purpose:** Produce the audio.
 - **Main functionality:** Calls Sarvam AI TTS (ElevenLabs as fallback); manages voice profiles and cloned voices with consent records; applies a per-project pronunciation lexicon for technical terms; caps rate adjustment at $\pm 10\%$.
4. **Module 4 - Timeline & Sync**
 - **Purpose:** Keep narration and on-screen action aligned.
 - **Main functionality:** Force-aligns generated audio to text; anchors segments to action cues; allocates residual slack as holds at non-action gaps; measures per-segment drift and mean rate; emits the QA report.
5. **Module 5 - Render & Export**
 - **Purpose:** Produce deliverables the Client can actually use.
 - **Main functionality:** FFmpeg compositing and muxing; browser preview proxy; export to OpenTimelineIO / Kdenlive XML, per-segment WAV stems and SRT subtitles.
6. **Module 6 - Project Workspace and Review UI**
 - **Purpose:** Give humans control over every automated decision.
 - **Main functionality:** Project/segment editor, side-by-side translation review, preview player, approval workflow, decision log.
7. **Module 7 - Admin and Integrations**
 - **Purpose:** Operate the system safely.

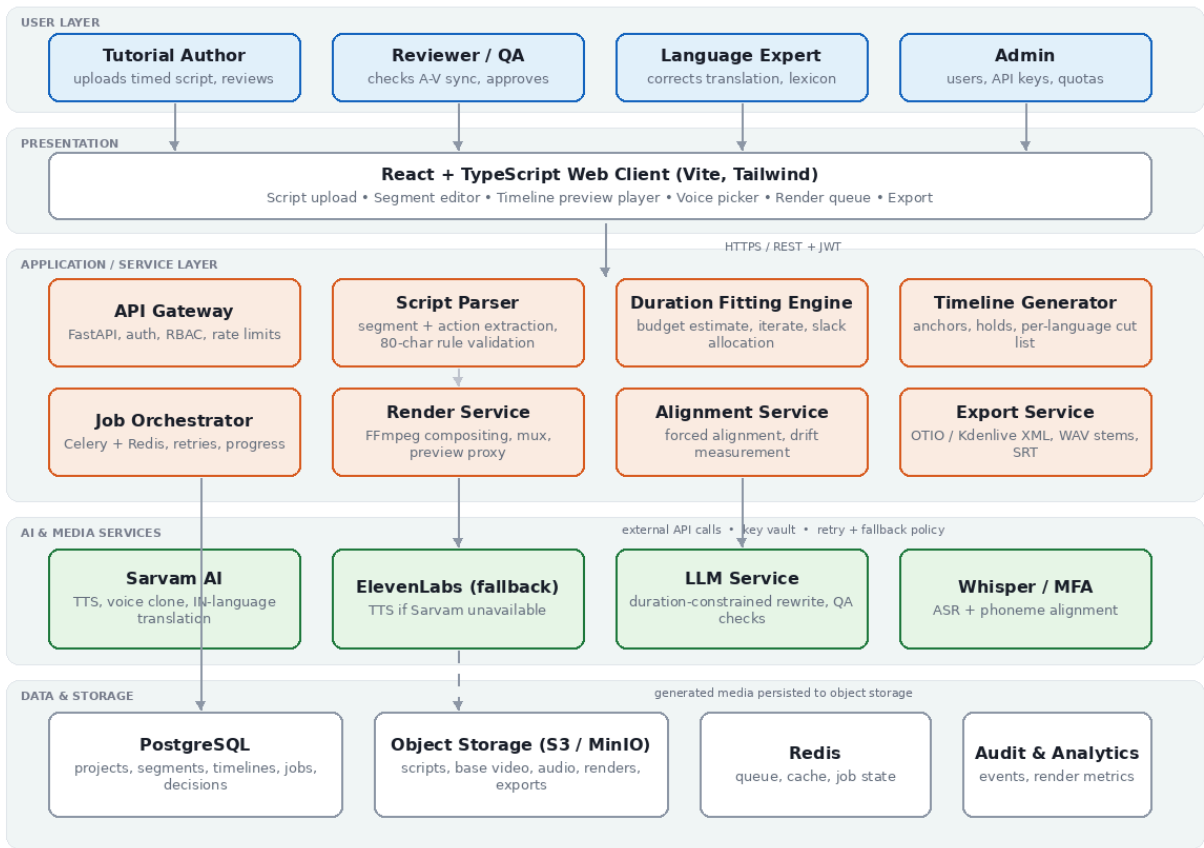
- **Main functionality:** User and role management, API key vault, per-project quota and usage reporting, provider switching.
-

6.2 Module Dependency



7. High-Level System Architecture

7.1 Architecture Diagram



7.2 Architecture Components

Component	Purpose	Technology / Proposed Technology
Frontend	Authoring workspace, segment editor, preview player, review and approval UI	React 18 + TypeScript, Vite, Tailwind CSS
Backend	REST API, orchestration of parsing, translation, synthesis, timeline and export	Python 3.14, FastAPI, Pydantic
Job queue	Asynchronous long-running media jobs with progress and retries	Celery + Redis

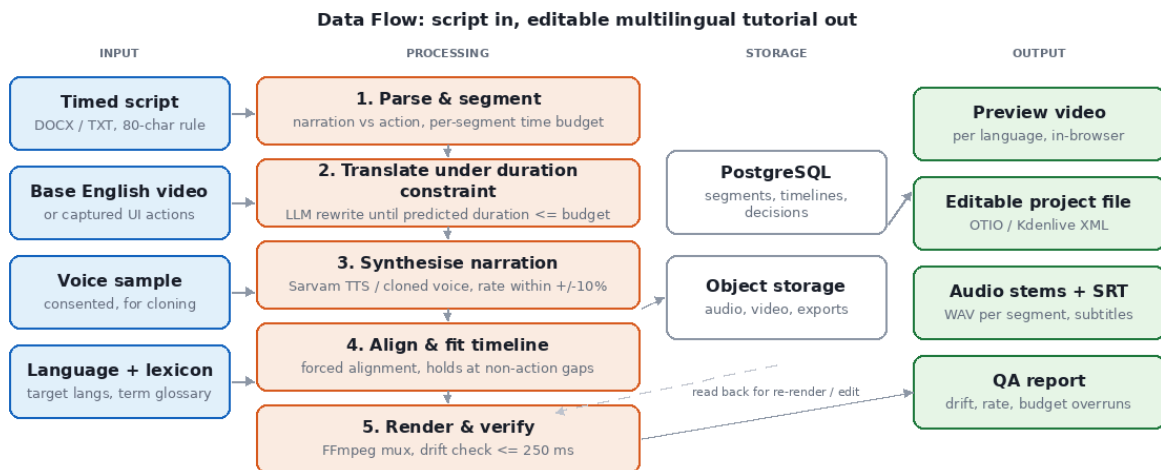
Database	Projects, segments, timelines, voice profiles, jobs, approvals, decision log	PostgreSQL 15
Object storage	Scripts, base video, generated audio, renders, exports	S3-compatible (MinIO on institutional server, or AWS S3)
AI - TTS & voice clone	Narration synthesis in Indian languages; voice cloning	Sarvam AI (primary, per Client partnership); ElevenLabs (fallback)
Media processing	Compositing, hold insertion, muxing, proxy generation	FFmpeg
Authentication	Login, session management, role-based authorisation	JWT with refresh tokens; bcrypt password hashing
Hosting	Application and worker deployment	Docker Compose deployment; Hugging Face Spaces for the demo instance
Version control / CI	Source management and reproducible builds	GitHub, GitHub Actions

7.3 External Services

Service	Purpose	Dependency
Sarvam AI	TTS, voice cloning, Indian-language translation support	High - named by the Client as the target provider
ElevenLabs	Fallback TTS while Sarvam capabilities are validated	Medium - currently in use by the Client
LLM API	Duration-constrained translation and re-phrasing	High - no equivalent offline substitute within project scope
WhisperX / MFA	Forced alignment for drift measurement	Medium - self-hostable; can run offline
Object storage provider	Media artefact persistence	Medium - substitutable with local MinIO

8. Data Architecture

8.1 Data Flow



8.2 Key Data Entities

Entity	Description	Key Information
User	An authenticated person using the tool	ID, name, institutional email, role, status
Project	One tutorial being produced	ID, title, software/topic, base language, owner, status
Script	An uploaded timed script version	ID, project, source file reference, version, parse status
ScriptSegment	One narration unit with its time budget	ID, script, order, source text, character count, start/end, budget (s), rule violations

8.3 Data Relationships

- Project → contains → Script (versioned)
- Script → is parsed into → ScriptSegment → anchors → ActionCue
- Project → has many → LanguageTrack

- LanguageTrack → contains → TranslatedSegment → generates → AudioAsset
 - LanguageTrack + AudioAsset + ActionCue → produce → TimelineItem
 - LanguageTrack → is evaluated by → QAReport → gates → approval and export
 - VoiceProfile → is used by → AudioAsset; VoiceProfile → requires → consent record
-

8.4 Data Storage

Data stored:

Scripts and their parsed segments; translated text and reviewer edits; generated audio, timelines, renders and exports; voice profiles and consent records; user accounts and roles; job history and QA metrics; the decision log

Data not stored:

No learner data of any kind - the tool never touches student records, assessment data or usage analytics from the Client's learning platform. No raw voice-actor recordings beyond the consented cloning sample. No third-party API keys in source control.

Storage location:

To be stored in AWS S3 and PostgreSQL. Generated media stays within the chosen region; only text is sent to external AI providers, plus the consented voice sample at the point of cloning

Retention:

Project artefacts retained for the duration of the project and handed over at closure. Intermediate render artefacts purged. Consent records retained for as long as the associated cloned voice exists

9. UI / UX Architecture

9.1 Screen Inventory

Screen	User	Purpose
Login	All	Authenticate and route to the role-appropriate landing view
Project Dashboard	Author, Admin	List projects, language track states and pending reviews
Script Workspace	Author	Upload script, review parsed segments, resolve rule violations, set budgets
Language & Voice Setup	Author	Select target languages, voice profiles and pronunciation lexicon
Translation Review	Language Reviewer, Author	Side-by-side source/translation with durations; per-segment re-synthesis
Timeline & Preview	Author, QA Reviewer	Preview the track, inspect holds and anchors, read the QA report
Render & Export	Author, QA Reviewer	Queue renders, monitor jobs, download project file, stems and subtitles
Admin Console	Admin	Users, roles, API keys, provider selection, quota and usage

9.2 Navigation Structure

Login → Project Dashboard → Script Workspace → Language & Voice Setup → Translation Review → Timeline & Preview → Render & Export

Navigation is linear by default because the pipeline is ordered, but every completed stage remains directly reachable from the project navigation rail, since correction cycles are the normal case rather than the exception.

9.3 Wireframes

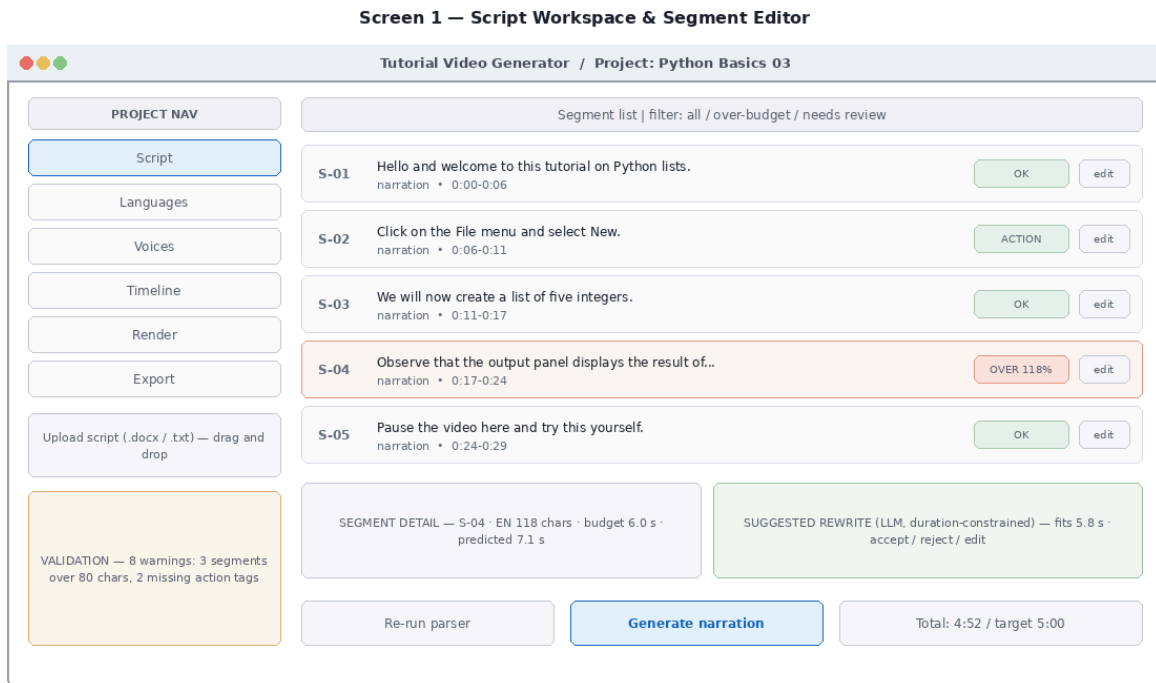
Include low-fidelity wireframes for the key screens.

Screen 1: [Script Workspace & Segment Editor]

Purpose: give the author a single view of every segment, its time budget, and whether it will fit - before any audio is generated

Primary actions:

- upload script; resolve validation warnings; edit a segment; accept or reject a duration-constrained rewrite; generate narration



Screen 2: Timeline, Preview & Sync Review

Purpose: Timeline, Preview & Sync Review

Primary actions:

- switch language track; play preview; inspect holds and anchors; read the QA report; approve or send back for correction

Screen 2 — Timeline, Preview & Sync Review

Timeline / Python Basics 03 / Tamil

VIDEO PREVIEW — base screen recording + generated narration

0:00 ————— 5:02 [play] [.5x] [1x]

LANGUAGE

English (base)5:00 • ref

Hindi5:02 • +0.7%

Tamil5:02 • +0.7%

Marathiqueued

SYNC REPORT — max drift 180 ms • rate 1.04x • 0 segments over limit

TIMELINE

VIDEO

clip 1clip 2clip 3clip 4clip 5clip 6

AUDIO

S-01S-02S-03S-04S-05S-06

HOLDS

holdholdhold

Holds are inserted only at non-action gaps, so click cues stay frame-accurate

10. Technology Stack

10.1 Frontend

Platform: Web (desktop browsers)

Framework: React 18 with Vite.

Language: TypeScript. Styling: Tailwind CSS. Media: HTML5 video with a generated low-bitrate preview proxy

10.2 Backend

Framework: FastAPI.

Language: Python 3.14.

API architecture: REST (JSON), with polling endpoints for job progress. Task queue: Celery with Redis broker

10.3 Database

Database: PostgreSQL 15 for relational project data (projects, segments, tracks, jobs, approvals, decision log). Redis for queue state and short-lived caching. S3-compatible object storage for all media artefacts

10.4 AI / ML Components

Model/service: Sarvam AI for TTS, voice cloning and Indian-language support; an LLM API for duration-constrained translation and re-phrasing; WhisperX or Montreal Forced Aligner for forced alignment.

Inference: cloud for TTS and LLM; alignment can run on the application server.

External API: Yes. Fallback: ElevenLabs for TTS; on repeated provider failure, the job is queued and the author is notified rather than the pipeline failing silently, and any segment that cannot be fitted automatically is escalated for manual authoring.

10.5 Infrastructure

Hosting: Hugging Face

Deployment: Docker Compose (api, worker, db, redis, object store)

Version control: GitHub, with GitHub Actions for lint, tests, and image builds. Secrets: environment-injected, never committed

11. Non-Functional Requirements

11.1 Performance

- Interactive UI response (navigation, segment edit, save): ≤ 500 ms at the 95th percentile.
- Script parsing for a typical 10-minute tutorial script: ≤ 10 s.
- Narration synthesis: ≤ 3 s per segment, subject to provider latency; a full track synthesised in parallel batches.
- End-to-end render of a 5-minute tutorial in one language: ≤ 10 minutes, executed asynchronously with visible progress.
- Maximum expected concurrent users: 10 authoring sessions; 3 concurrent render jobs per worker node.

11.2 Accessibility

- Target standard: WCAG 2.1 Level AA for the authoring interface.
- Keyboard accessibility: all authoring actions reachable without a pointer, including segment navigation and playback control.
- Screen reader support: semantic landmarks, labelled controls, ARIA live regions for job progress.
- Colour contrast: minimum 4.5:1 for text; status is never conveyed by colour alone - over-budget segments carry a text label as well as a colour.

11.3 Security

- Authentication: JWT access and refresh tokens; passwords hashed with bcrypt; session expiry on inactivity.
- Authorisation: role-based access control enforced server-side on every endpoint, per §4 - not merely hidden in the UI.
- Data protection: encryption in transit (TLS 1.2+); object storage encrypted at rest; signed, expiring URLs for media downloads.
- Secrets: third-party API keys held in an environment-injected vault, never in source control or client-side code; all provider calls proxied through the backend.
- Auditing: authentication events, approvals, exports, and provider calls logged with user and timestamp.

11.4 Privacy

- Personally identifiable information collected: Yes - limited to authoring-side accounts (name, institutional email, role) and voice-cloning consent records.

- **Student data collected:** None. The tool has no learner-facing surface and no connection to the Client's learner records.
- **Data anonymization:** Not applicable to learner data (none is held). Voice samples are linked only to a named, consenting voice artist.
- **Data sharing:** Script and translated text are sent to the LLM and TTS providers for processing. The consented voice sample is sent to Sarvam AI at the point of cloning. No data is shared with any other party.
- **Third-party AI data processing:** Provider data-retention and training-use terms to be reviewed and recorded before production use; the team will request the Client's written confirmation that script content may be sent to external providers.

11.5 Compatibility

Supported devices:

- desktop and laptop computers; recommended 1366×768 viewport.

Supported browsers / operating systems:

- Chrome, Edge- current and one prior major version. Safari on a best-effort basis
- Windows 10+, Ubuntu 22.04+, macOS 13+ (client side). Server side: Linux only.
- Output compatibility: H.264/AAC MP4 for renders; exported project files validated against Kdenlive and OpenTimelineIO.

11.6 Offline Requirements

The authoring tool requires connectivity: translation, synthesis and rendering are all network- or server-dependent, and no offline mode is in scope. Two mitigations are in place. First, previously generated artefacts (audio, renders, exports) remain downloadable and usable offline, so a completed track is never stranded behind the tool. Second, a queued render survives a client disconnection - the job continues server-side and the author picks it up on reconnection. The Client's learners, who do face connectivity constraints, consume the exported files through existing channels and are unaffected by this.

12. Assumptions & Constraints

12.1 Assumptions

1. The Client will supply representative Spoken Tutorial scripts and sample videos (including at least one Tamil example exhibiting the rushed-narration problem) via the shared Drive folder, by the date
2. Scripts follow the documented Spoken Tutorial pedagogy - short sentences, the 80-character rule in English, one instruction per sentence - with sufficient consistency to be parsed programmatically.
3. The Client will provide Sarvam AI API access, or credentials sufficient for development and testing, within the project timeline.
4. The base visual track for a tutorial is authored once in English and reused across languages, as described by the Client.
5. A native-speaker reviewer will be available to judge narration quality in at least one Indian language; automated metrics alone cannot establish that narration sounds unrushed.
6. The Client accepts an editable project file as a valid deliverable, rather than requiring only a final rendered video.

12.2 Constraints

1. Academic timeline: MVP due by week 8 and the team carrying a concurrent course load.
 2. Team composition: four final-year students, none of whom is a full-time engineer on this project; effort is bounded accordingly.
 3. Third-party dependency: narration quality is bounded by what Sarvam AI produces for a given language. The team can control pacing, phrasing, lexicon and alignment; it cannot control the provider's underlying voice naturalness - a limitation the team has already observed with regional-language synthesis.
 4. Cost: LLM and TTS calls are metered. Development uses short, representative scripts rather than full-length tutorials to stay within quota.
 5. The video track is treated as an input. Automatic generation of screen-recorded software demonstrations from a script is out of scope for this project
 6. Language coverage during development is limited to English plus two Indian languages, at least one of which is Tamil, since Tamil is the Client's worst case.
-

13. Project Scope

13.1 In Scope

The following features are included in the project:

- Script upload and parsing into narration segments, action cues and per-segment time budgets, with pedagogy-rule validation.
- Duration-constrained translation: predicted-duration estimation and iterative re-phrasing so the translation fits the budget.
- Narration synthesis via Sarvam AI, including voice cloning from a consented sample, with ElevenLabs as fallback.
- A per-project pronunciation lexicon for technical terms, applied consistently across a tutorial series.
- Forced alignment, per-segment drift and speaking-rate measurement, and an objective QA report per language track.
- Timeline generation with hold insertion restricted to non-action gaps, preserving frame-accurate action cues.
- In-browser preview of each language track; segment-level correction and re-synthesis.
- Export of an editable project file (OpenTimelineIO / Kdenlive XML), per-segment WAV stems, SRT subtitles and a rendered MP4.
- Role-based access control, approval workflow and a decision log.
- Support for English plus two Indian languages, at least one being Tamil.

13.2 Out of Scope

The following are explicitly excluded from the project:

- Automatic generation of the screen-recording itself - that is, driving arbitrary third-party software from a script to produce the visual demonstration. The base video is an input.
- Direct publishing to the Client's LMS or website, and any integration with learner records or assessment data.
- A mobile or desktop-native application; the tool is desktop web only.
- Real-time multi-user collaborative editing of the same project.
- Full replacement of the Client's video editor; the tool produces editable projects, it is not an editor.
- Training or fine-tuning of custom TTS models; the project consumes provider APIs.
- Gamification of the Client's Python testing process
- Production-grade scaling, SLA-backed uptime, or commercial deployment and support.

14. Acceptance Criteria

The application will be considered functionally complete when:

Core Functionality

- A Client-supplied timed script is parsed into segments and action cues with $\geq 95\%$ segment-boundary accuracy across at least 10 sample scripts, with all rule violations correctly flagged.
- A complete language track is generated end to end from script to export for English plus at least two Indian languages, one of them Tamil.
- Voice cloning from a consented sample is demonstrated for at least one language, and the system refuses to register a voice profile without a consent record.
- A single-segment correction regenerates only that segment and updates the timeline and QA report without a full re-render.

User Experience

- Per-segment audio–video drift ≤ 250 ms across the full tutorial, measured by forced alignment and reported automatically.
- Mean speaking-rate ratio within $\pm 10\%$ of the reference pace for every language track, with no segment exceeding $\pm 15\%$.
- Holds are inserted only at non-action gaps; every action cue remains aligned to its narration within the drift threshold.
- A native-speaker reviewer rates the generated Tamil narration as no more rushed than the Client's current output, on a Client-agreed rating scale.

Technical

- Exported project files open in Kdenlive without manual repair, with narration on separate tracks and holds preserved.
- A 5-minute tutorial renders in ≤ 10 minutes, asynchronously, with visible progress and safe recovery from a client disconnection.
- Role-based access control enforced server-side; approval cannot be performed by the user who generated the track.
- Source code in a GitHub repository with setup documentation sufficient for a third party to run the system.

Deployment

- A working instance is deployed and reachable by the Client for review, with a documented deployment procedure and a seeded demo project.
-

15. Development Dependencies

Dependency	Required From	Required By	Status
Sample Spoken Tutorial videos (incl. a Tamil example showing the pacing problem)	Client	Meeting 2 - 18 Aug 2026	Received
Corresponding timed scripts in source format (.docx/.txt)	Client	18 Aug 2026	Received
Written statement of the pedagogy rules (sentence length, structure, timing convention)	Client	22 Aug 2026	Pending
Sarvam AI API access or test credentials	Client	25 Aug 2026	Pending
Confirmed target language list for the project	Client	22 Aug 2026	Pending
Voice sample plus signed consent for cloning	Client	5 Sep 2026	Pending
Named language reviewer for quality judgement	Client	5 Sep 2026	Pending
Approval to send script text to external AI providers	Client	25 Aug 2026	Pending
Hosting decision and deployment credentials	Client / Course team	3 Oct 2026	Pending
HLD review and sign-off	Client	22 Aug 2026	Pending

16. Open Questions & Decision Log

ID	Question / Decision	Decision	Owner / Date
D-01	Which target languages are in scope for development and evaluation?	Team proposes English + Tamil + one more (Hindi or Marathi).	Client - Meeting 3
D-02	May script content be sent to external LLM and TTS providers, and under what retention terms?	Pending - team to review provider terms and present them.	Client - Meeting 3
D-03	Does Sarvam AI support voice cloning at the required quality for Tamil?	Pending - Client to arrange a demonstration with the internal team already working on Sarvam.	Client - Meeting 3
D-04	Where will project data and generated media be hosted?	Pending.	Client - Meeting 3
D-05	What rating scale will be used to judge whether narration sounds unrushed?	Team proposes a 5-point native-speaker rating on naturalness and pace, scored against the Client's current output as the baseline.	Client - Meeting 3

17. Project & HLD Agreement

The Student Team and the Client confirm that they have reviewed and discussed the project requirements, proposed solution, user personas, workflows, system design, project scope, technical approach, and deliverables described in the **High-Level Design (HLD) Document**.

The HLD, together with this Agreement, forms the agreed basis for the development of the educational application.

17.1 Agreement on HLD

Both parties agree that:

- The **High-Level Design (HLD) Document** represents the current agreed understanding of the project.
- The HLD defines the intended **users, user journeys, core functionality, system architecture, technology approach, scope, assumptions, constraints, and acceptance criteria**.
- The Student Team will use the approved HLD as the basis for designing and developing

the application.

- The Client will review the HLD and provide feedback or approval within the agreed project schedule.
- Any **significant change to the agreed scope or functionality** will be discussed between the Student Team and Client before implementation.
- Changes may affect the project timeline and deliverables and will be incorporated only after mutual agreement.

17.2 Agreement on Deliverables and Timeline

The Student Team agrees to work towards completing the project deliverables within the agreed project timeline specified in **Section 3: Project Timeline & Deliverables**.

The Student Team will:

- Complete the agreed development work for each milestone.
- Present the agreed deliverables to the Client for review.
- Incorporate reasonable feedback provided within the agreed review period.
- Provide the final deliverables specified in **Section 4: Final Deliverables**.
- Maintain the project source code and development documentation throughout the project.

The Client agrees to:

- Provide the information, content, data, access, credentials, and other materials required for development.
- Attend the agreed project meetings or provide an appropriate representative.
- Review milestone deliverables within the specified feedback period.
- Provide clear and consolidated feedback to the Student Team.
- Communicate any major changes to requirements as early as possible.

17.3 Timeline Dependencies

The agreed development timeline is dependent on the timely participation and feedback of both parties.

If required information, content, access, approvals, or feedback from the Client is delayed, the corresponding development milestone may need to be revised.

Similarly, if the Student Team identifies a technical or implementation issue that materially affects the agreed timeline, the team will communicate the issue to the Client as soon as reasonably possible.

17.4 Final Deliverables

At the conclusion of the project, the Student Team will provide the deliverables agreed upon in this document and the approved HLD, subject to the project's scope and timeline.

These include, as applicable:

- **Executable Application:** Final working educational application/prototype.
- **Source Code:** Complete project source code and repository.
- **User Documentation:** Instructions for using the application.
- **Technical Documentation:** High-level architecture, setup and technical information.
- **Project Presentation:** Final presentation of the project and outcomes.

Week	Dates	Key Tasks & Milestones	Milestone Deliverable / HLD Reference	Client Feedback Due
2	7 – 14 Aug	Requirements gathering, client meeting, HLD authoring	HLD Draft (this document)	18 Aug 2026
3	15 – 22 Aug	HLD finalisation, UI/UX wireframes, sample script and video analysis	Approved HLD + wireframes (§9.3)	22 Aug 2026
4 – 6	23 Aug – 6 Sep	Core development: script parser, duration model, TTS integration	Milestone 1 - script → segmented timeline → narration for one language (§6 M1–M3)	10 Sep 2026
7 – 8	7 – 24 Sep	Duration-fitting engine, alignment, hold insertion, preview UI	MVP - end-to-end English + one Indian language with QA report (§14)	24 Sep 2026

17.5 Mutual Confirmation

By signing below, both parties confirm that:

We have reviewed and discussed the project requirements and High-Level Design (HLD), agree to use the approved HLD as the basis for development, and commit to working collaboratively towards completing the agreed deliverables within the project timeline.

Both parties understand that timely communication, access to required information, feedback, approvals, and participation in project meetings are necessary to meet the

agreed timeline.

Signatures

Student Team

Student 1 Name (Roll No): _____ 23B2503 _____

Signature: _____ Daksh Jain _____

Date: _____ 19-08-2026 _____

Student 2 Name (Roll No): _____ 23B2509 _____

Signature: _____ Sameer Gupta _____

Date: _____ 19-08-2026 _____

Student 3 Name (Roll No): _____ 23B2487 _____

Signature: _____ Sumedh Taru _____

Date: _____ 19-08-2026 _____

Student 4 Name (Roll No): _____ 23B2244 _____

Signature: _____ Sanidhya Katiyar _____

Date: _____ 19-08-2026 _____

Client

Client Name: _____ Saisudha _____

Organization: _____ EduPyramids, IIT Bombay _____

Signature: _____

Date: _____

HLD Approval

HLD Document Version: _____ 1.0 _____

HLD Approval Date: _____